

Merchant Intelligence Dashboard

Complete frontend page catalog from onboarding to audit

Product goal

The frontend should help a merchant answer, in order:

Plain Text

What did I expect to collect?
What did Razorpay observe?
What was settled?
What reached my bank?
What is missing or inconsistent?
What should I review?
What money is coming and going next?
What does the AI know, and what evidence supports it?

The frontend is not a copy of the Razorpay Dashboard. It is a merchant-control layer that combines merchant orders, payment intents, Razorpay payments, webhooks, settlement reconciliation, bank statements, evidence, ML predictions, and AI explanations.

1. Global application shell

Route behavior

All authenticated merchant pages use:

Plain Text

/merchant/*

Global layout

Area	Functionality
Top navigation	Merchant name, environment badge, date range, currency, notifications, user menu.
Side navigation	Overview, Imports, Transactions, Intents, Settlements, Gaps, Outcomes, Intelligence,

	Forecasts, Ask, Evidence, Connectors, Settings.
Environment badge	Clearly shows <code>Test Mode</code> or <code>Live Mode</code> ; never mix data silently.
Freshness badge	Displays latest webhook, API backfill, settlement, and bank-import timestamps.
Global search	Search by intent ID, order ID, payment ID, settlement ID, UTR, bank reference, or gap ID.
Source badges	Shows <code>Intent</code> , <code>Razorpay Webhook</code> , <code>Razorpay API</code> , <code>Settlement Recon</code> , <code>Bank CSV</code> , <code>Derived</code> , or <code>Forecast</code> .
Notifications	Connector errors, failed imports, stale data, high-severity gaps, model failures, and review assignments.

Global states

Every page must support `loading` , `success` , `empty` , `stale` , `partial_data` , `permission_denied` , `connector_not_ready` , `retrying` , and `api_error` .

AI/ML pages also need `insufficient_history` , `low_confidence` , `model_stale` , and `explanation_unavailable` .

2. Page 0 — Sign-in and merchant onboarding

Routes

Plain Text

`/login`
`/onboarding`

Purpose

Create or access a merchant tenant, then configure the first Razorpay Test Mode connector.

Functionality

The page should collect merchant name, business type, base currency, timezone, and data-source preference. The user can choose Razorpay Test Mode, configure a webhook

endpoint, or begin with CSV-only data.

Backend APIs

Plain Text

```
POST /api/v1/auth/login
GET  /api/v1/merchant/profile
POST /api/v1/merchant/onboarding
GET  /api/v1/merchant/onboarding/status
```

Output

JSON

```
{
  "data": {
    "tenant_id": "tenant_001",
    "merchant_name": "Demo Merchant",
    "onboarding_status": "connector_required",
    "recommended_next_route": "/connectors/new"
  }
}
```

3. Page 1 — Overview / Merchant Dashboard

Route

Plain Text

/dashboard

Purpose

Give the merchant a daily summary of money coming in, money going out, what has been reconciled, what is missing, and what is expected next.

Main cards

Plain Text

Gross payment volume
Captured amount
Razorpay settled amount
Bank-credited amount
Unsettled amount
Open payment gaps
Amount at risk
Next 7-day net cash forecast

Charts

Plain Text

Daily incoming versus outgoing cash
Captured → settled → bank-matched funnel
Settlement delay trend
Open-gap aging
Refund and dispute trend
Forecast with confidence band

Actions

Plain Text

Open high-priority gaps
Run backfill
Upload bank statement
View today's AI briefing
Open outcome review
Ask Ask Zord

Backend API

Plain Text

GET /api/v1/merchant/dashboard/summary
GET /api/v1/merchant/dashboard/trends
GET /api/v1/merchant/dashboard/actions

Key response

JSON

```
{
  "data": {
    "incoming": {
      "gross": 125000000,
      "captured": 118000000,
      "settled": 114420000,
      "bank_credited": 113870000
    },
    "outgoing": {
      "refunds": 3200000,
      "transfers": 1250000,
      "fees_and_tax": 333000
    },
    "reconciliation": {
      "fully_reconciled": 1240,
      "provider_settled_bank_pending": 18,
      "captured_settlement_pending": 31,
      "amount_mismatch": 7,
      "unresolved": 4
    },
    "forecast": {
      "next_7_days_incoming": 28200000,
      "next_7_days_outgoing": 9100000,
      "next_7_days_net": 19100000,
      "confidence": 0.81
    },
    "freshness": {
      "status": "fresh",
      "last_provider_sync": "2026-08-29T10:25:00Z",
      "last_bank_import": "2026-08-28T18:00:00Z"
    }
  }
}
```

4. Page 2 — Data Imports

Route

Plain Text

/data-imports

Purpose

Upload, validate, map, commit, and monitor transaction files, settlement files, bank statements, order files, refund files, and optional dispute reports.

Upload cards

Plain Text

```
Upload transaction CSV
Upload settlement CSV
Upload bank statement CSV
Upload order/intent CSV
Upload refund CSV
Upload dispute report
```

Workflow

Plain Text

```
Select file type
→ Upload file
→ Detect columns
→ Map columns
→ Preview rows
→ Validate
→ Review errors
→ Commit
→ Start reconciliation
→ Show results
```

APIs

Plain Text

```
POST /api/v1/merchant/imports/upload-url
POST /api/v1/merchant/imports
GET  /api/v1/merchant/imports
GET  /api/v1/merchant/imports/:id
POST /api/v1/merchant/imports/:id/validate
POST /api/v1/merchant/imports/:id/commit
POST /api/v1/merchant/imports/:id/cancel
```

Required UI states

Plain Text

awaiting_upload
uploading
uploaded
mapping_required
validating
validation_failed
validated
committing
committed
partial
failed

Output

Show rows seen, valid rows, invalid rows, duplicate rows, inserted records, updated records, rejected records, response hash, and reconciliation job ID.

5. Page 3 — Transactions list

Route

Plain Text

/transactions

Purpose

Show the normalized payment ledger across merchant intent, Razorpay webhook, Razorpay API, settlement recon, and bank sources.

Filters

Plain Text

Date range
Payment status
Payment method
Amount range
Currency
Order ID
Payment ID
Source presence
Settlement state

Bank-credit state
Gap status

Table columns

Plain Text

Merchant reference
Intent ID
Razorpay order ID
Razorpay payment ID
Gross amount
Payment status
Settlement status
Bank status
Gap status
Created time
Last observed time

Source chips

Plain Text

[Intent] [Webhook] [API] [Settlement] [Bank] [Verified]

APIs

Plain Text

GET /api/v1/merchant/transactions
GET /api/v1/merchant/transactions/export
POST /api/v1/merchant/transactions/bulk-review

Row actions

Plain Text

Open details
Open evidence
Open payment gap
Request backfill

Add review note
Export selected

6. Page 4 — Transaction detail and proof

Route

Plain Text

```
/transactions/:transaction_id
```

Purpose

Show the complete payment-to-bank journey for one transaction.

Layout

Plain Text

Header: amount, state, payment ID, order ID

Left: evidence timeline

Center: state progression

Right: source records and actions

Bottom: reconciliation calculation and audit metadata

State progression

Plain Text

Intent created

→ Payment authorized

→ Payment captured

→ Settlement observed

→ Bank credit matched

→ Reconciliation verified

APIs

Plain Text

```
GET /api/v1/merchant/transactions/:id
```

```
GET /api/v1/merchant/transactions/:id/proof
```

```
GET /api/v1/merchant/transactions/:id/evidence
GET /api/v1/merchant/transactions/:id/source-records
```

Proof display

Plain Text

```
Payment captured: Proven
Provider settlement: Proven
Bank credit: Proven
Overall reconciliation: Verified
Match rule: Exact UTR + exact amount
Evidence root: merkle_root_001
```

The page must clearly distinguish `Razorpay settled` from `Bank credited` .

7. Page 5 — Payment Intents and Orders

Routes

Plain Text

```
/intents
/intents/:intent_id
```

Purpose

Show what the merchant expected to collect and connect it to provider execution.

Functionality

Plain Text

```
Expected amount and currency
Merchant order reference
Customer/cart reference
Provider order ID
Payment attempts
Captured result
Settlement result
Final bank result
```

APIs

Plain Text

```
GET /api/v1/merchant/intents
GET /api/v1/merchant/intents/:id
GET /api/v1/merchant/intents/:id/attempts
POST /api/v1/merchant/intents/import
```

Detail states

Plain Text

```
created
payment_pending
authorized
captured
partially_paid
paid
failed
expired
refunded
```

8. Page 6 — Settlements list

Route

Plain Text

```
/settlements
```

Purpose

Show Razorpay settlement batches and bank matching state.

Table columns

Plain Text

```
Settlement ID
Settlement date
UTR
Gross payment amount
```

Refunds
Transfers
Fees
Tax
Adjustments
Expected net
Bank credit
Difference
Status

APIs

Plain Text

GET /api/v1/merchant/settlements
GET /api/v1/merchant/settlements/export
POST /api/v1/merchant/settlements/:id/request-repair

Statuses

Plain Text

expected
partially_observed
settled
bank_pending
bank_matched
amount_mismatch
on_hold
adjusted

9. Page 7 — Settlement detail and waterfall

Route

Plain Text

/settlements/:settlement_id

Purpose

Explain exactly how gross payments became expected net settlement and whether that net amount reached the bank.

Waterfall

Plain Text

```
Payment credits
- Refund debits
- Transfer debits
- Fees
- Tax
+ Adjustments
= Expected net settlement
→ Bank credit
→ Difference
```

APIs

Plain Text

```
GET /api/v1/merchant/settlements/:id
GET /api/v1/merchant/settlements/:id/lines
GET /api/v1/merchant/settlements/:id/evidence
GET /api/v1/merchant/settlements/:id/bank-match
```

Actions

Plain Text

```
Open payment lines
Open refund/transfer/adjustment lines
View UTR
View bank match
Request API backfill
Open evidence
Create review item
```

10. Page 8 — Payment Gaps

Routes

Plain Text

/payment-gaps

/payment-gaps/:gap_id

Purpose

Show every unresolved link or inconsistency in the payment-to-bank chain.

Gap types

Plain Text

intent_without_payment

payment_without_intent

payment_without_webhook

payment_without_settlement

settlement_without_bank_credit

bank_credit_without_settlement

amount_mismatch

fee_tax_mismatch

duplicate_provider_record

out_of_order_event

stale_observation

schema_error

Table columns

Plain Text

Gap ID

Gap type

Severity

Payment/order ID

Settlement ID

Expected amount

Observed amount

Difference

Age

Confidence

Owner

Status

APIs

Plain Text

```
GET /api/v1/merchant/payment-gaps
GET /api/v1/merchant/payment-gaps/:id
POST /api/v1/merchant/payment-gaps/:id/assign
POST /api/v1/merchant/payment-gaps/:id/request-backfill
POST /api/v1/merchant/payment-gaps/:id/resolve
POST /api/v1/merchant/payment-gaps/:id/reopen
```

Gap detail visualization

Plain Text

[Intent] ✓ → [Payment] ✓ → [Settlement] ✓ → [Bank credit] ?

The detail view must show the missing signal, last attempted repair, evidence available, recommended next action, and whether approval is required.

11. Page 9 — Outcome Review queue

Routes

Plain Text

```
/outcome-review
/outcome-review/:outcome_id
```

Purpose

Give finance operators a human review queue for ambiguous, high-value, stale, or mismatched outcomes.

Queue filters

Plain Text

```
Severity
Age
Gap type
Amount at risk
Assigned user
Status
```

Confidence
Provider

Detail functionality

Plain Text

Evidence timeline
Expected-versus-observed calculation
Candidate matches
AI explanation
Recommended action
Operator notes
Assignment
Resolve/reopen controls
Immutable resolution history

APIs

Plain Text

GET /api/v1/merchant/outcomes/review-queue
GET /api/v1/merchant/outcomes/:id
POST /api/v1/merchant/outcomes/:id/claim
POST /api/v1/merchant/outcomes/:id/add-note
POST /api/v1/merchant/outcomes/:id/resolve
POST /api/v1/merchant/outcomes/:id/reopen

12. Page 10 — Reconciliation Control Center

Route

Plain Text

/reconciliation

Purpose

Provide an operational overview of reconciliation jobs, source freshness, matching rates, and repair activity.

Panels

Plain Text

```
Last webhook receipt
Last payment backfill
Last settlement backfill
Last bank import
Records awaiting match
Matched records
Ambiguous matches
Failed jobs
Cursor positions
```

APIs

Plain Text

```
GET /api/v1/merchant/reconciliation/summary
GET /api/v1/merchant/reconciliation/jobs
GET /api/v1/merchant/reconciliation/freshness
POST /api/v1/merchant/reconciliation/run
```

This page is valuable for judges because it shows that the system is not only a dashboard; it is an operating control loop.

13. Page 11 — Intelligence Center

Route

Plain Text

```
/intelligence
```

Purpose

Combine deterministic KPIs, anomaly detection, operational recommendations, and model explanations.

Sections

Plain Text

Payment success and failure
Settlement delay analysis
Fee/tax variance
Refund pressure
Missing webhook rate
Bank-match reliability
Anomaly clusters
Merchant baseline changes
Daily AI briefing
Recommended actions

APIs

Plain Text

```
GET /api/v1/merchant/intelligence/overview
GET /api/v1/merchant/intelligence/anomalies
GET /api/v1/merchant/intelligence/briefing
GET /api/v1/merchant/intelligence/recommendations
POST /api/v1/merchant/intelligence/recommendations/:id/feedback
```

Model outputs

JSON

```
{
  "anomaly_id": "anom_001",
  "type": "settlement_delay",
  "score": 0.86,
  "severity": "high",
  "reason_codes": ["delay_above_baseline", "bank_credit_missing"],
  "coverage": "covered",
  "source_record_ids": ["setl_001", "bank_gap_001"],
  "recommended_action": "review_bank_statement_or_request_repair"
}
```

Use the existing FORGE model governance principles: keep detector, explanation, and generation roles separate; show model version; show confidence and coverage; and expose limitations.

14. Page 12 — Cash Forecast

Route

Plain Text

/forecasts/cash

Purpose

Forecast money entering and leaving the merchant's bank position.

Inputs

Plain Text

Historical captured payments
Expected settlement schedule
Observed settlement delays
Bank credits
Refunds
Transfers
Fees and tax
Recurring obligations, if available

Charts

Plain Text

Expected incoming cash
Expected outgoing cash
Net cash position
Lower/upper confidence band
Captured but unsettled amount
Settlement pipeline by expected credit date

APIs

Plain Text

GET /api/v1/merchant/forecasts/cash
GET /api/v1/merchant/forecasts/cash/explanations
GET /api/v1/merchant/forecasts/cash/scenarios

Required behavior

If data is insufficient, show `insufficient_history` . Forecasts must show confidence and intervals, not a false exact number.

15. Page 13 — Order and collection forecast

Route

Plain Text

`/forecasts/orders`

Purpose

Forecast future order volume, paid orders, captured amount, refunds, and likely collections.

Functionality

Plain Text

Expected orders by day
Expected paid orders
Expected conversion rate
Expected captured amount
Expected refund pressure
Channel and payment-method forecast
Feature explanations
Scenario comparison

APIs

Plain Text

GET `/api/v1/merchant/forecasts/orders`
GET `/api/v1/merchant/forecasts/orders/by-channel`
GET `/api/v1/merchant/forecasts/orders/explanations`

16. Page 14 — Ask Zord / Merchant AI assistant

Route

Plain Text

/ask

Purpose

Answer merchant finance questions using structured live data, evidence, RAG, and controlled action contracts.

Example questions

Plain Text

Why is this settlement not matched?
Which payments were captured but not settled?
Did this payment reach the bank?
What money is expected in the next seven days?
Which fees caused the largest variance?
Show all Razorpay-only records that have no webhook.
What should I review today?

APIs

Plain Text

POST /api/v1/merchant/ask
GET /api/v1/merchant/ask/sessions
GET /api/v1/merchant/ask/sessions/:id

Answer requirements

Every material statement must cite transaction, settlement, bank, gap, or forecast records.
The AI must distinguish:

Plain Text

provider settled
bank credited
fully reconciled
forecasted
probable
unresolved

Tool layer

Plain Text

```
get_dashboard_summary
search_transactions
get_transaction_proof
get_settlement_breakdown
get_bank_match
search_payment_gaps
get_forecast
get_freshness
create_backfill_request
assign_gap
```

The AI should propose action contracts instead of directly performing financial operations.

17. Page 15 — Audit Evidence Explorer

Route

Plain Text

```
/evidence
/evidence/:evidence_id
```

Purpose

Allow the merchant or auditor to inspect the source chain behind a financial conclusion.

Display

Plain Text

```
Source type
Source record ID
Raw payload hash
Observed timestamp
Normalized record ID
Match rule
Reconciliation decision
Evidence pack ID
```

Merkle root
Verification status

APIs

Plain Text

```
GET /api/v1/merchant/evidence
GET /api/v1/merchant/evidence/:id
GET /api/v1/merchant/evidence/:id/verify
GET /api/v1/merchant/evidence/export
```

Actions

Plain Text

```
Verify evidence
View redacted source payload
Download evidence pack
Copy record ID
Open related transaction
Open related gap
```

18. Page 16 — Connectors and data freshness

Route

Plain Text

```
/connectors
/connectors/new
/connectors/:connector_id
```

Purpose

Configure Razorpay Test/Live Mode, webhook status, API status, backfill status, bank import status, and connector health.

Functionality

Plain Text

```
Add Razorpay connector
Choose Test or Live mode
Store API-key and webhook-secret references
Run connection test
Configure webhook endpoint
View last API sync
View last webhook
View rate-limit/error status
Rotate secret reference
Disable connector
```

Never display secret values. Show only masked metadata and secret-reference names.

APIs

Plain Text

```
GET  /api/v1/merchant/connectors
POST /api/v1/merchant/connectors
GET  /api/v1/merchant/connectors/:id
POST /api/v1/merchant/connectors/:id/test
POST /api/v1/merchant/connectors/:id/rotate
POST /api/v1/merchant/connectors/:id/disable
```

19. Page 17 — Backfill and polling operations

Route

Plain Text

```
/operations/backfills
/operations/backfills/:job_id
```

Purpose

Show payment and settlement API backfill jobs, cursors, page progress, retries, and freshness impact.

Table columns

Plain Text

Job ID
Job type
Window
Status
Fetched
Inserted
Updated
Skipped duplicates
Missing webhooks
API errors
Cursor
Started
Finished

APIs

Plain Text

```
GET /api/v1/merchant/backfills
GET /api/v1/merchant/backfills/:id
POST /api/v1/merchant/backfills
POST /api/v1/merchant/backfills/:id/retry
POST /api/v1/merchant/backfills/:id/cancel
```

20. Page 18 — Refunds and disputes

Routes

Plain Text

```
/refunds
/refunds/:refund_id
/disputes
/disputes/:dispute_id
```

Purpose

Add these pages when the merchant uses refunds or disputes materially. Refunds affect outgoing cash and settlement lines. Disputes affect risk, reserve, and review workload.

Refund functionality

Plain Text

Full/partial refund status

Payment relationship

Refund amount

Settlement impact

Refund timeline

Dispute functionality

Plain Text

Open/lost/won state

Respond-by date

Amount at risk

Evidence status

Related payment/order

21. Page 19 — Payment Links, invoices, and subscriptions

Routes

Plain Text

/payment-links

/invoices

/subscriptions

Purpose

These are optional product-specific pages. Add them only if the merchant uses these Razorpay products. They support expected collections and recurring cash forecasts but should not delay the core reconciliation MVP.

22. Page 20 — Settings and access control

Routes

Plain Text

/settings

/settings/team

/settings/policies
/settings/notifications

Functionality

Plain Text

Merchant profile
Timezone and currency
Team members
Role permissions
Approval thresholds
Gap severity policies
Forecast preferences
Notification rules
Data retention
Audit export settings

23. Frontend-to-backend service ownership

Frontend area	Backend owner
Dashboard	Edge read API + outcome/intelligence read models.
Imports	Edge upload API + import worker + outcome normalizers.
Transactions/intents	Outcome-engine read models.
Settlements	Outcome-engine settlement store and recon read models.
Payment gaps	Outcome-engine variance/gap projections.
Outcome review	Outcome-engine plus agent action contracts.
Evidence	Evidence service.
Intelligence	Intelligence projections and ML service.
Forecasts	ML service and forecast read model.
Ask	Prompt layer, structured tools, RAG, and agent service.

Connectors	Edge connector handler and secret-reference manager.
Backfills	Outcome-engine poller and Airflow job APIs.

24. Recommended implementation order

MVP/P0

Plain Text

1. Global shell and authenticated merchant context.
2. Connectors page for Razorpay Test Mode.
3. Data Imports page.
4. Transactions list/detail.
5. Intents list/detail.
6. Settlements list/detail.
7. Payment Gaps list/detail.
8. Overview dashboard.
9. Evidence timeline on transaction and settlement detail.

P1

Plain Text

10. Reconciliation Control Center.
11. Outcome Review.
12. Intelligence Center.
13. Cash Forecast.
14. Ask Zord with citations.
15. Audit Evidence Explorer.
16. Backfill operations.

P2

Plain Text

17. Order Forecast.
18. Refunds.
19. Disputes.
20. Payment Links.

21. Invoices and subscriptions.
22. Advanced policies, roles, and notifications.

25. Complete judge demonstration

Plain Text

1. Open Connectors and show Razorpay Test Mode connected.
2. Open Data Imports and upload a transaction CSV.
3. Validate and commit the file.
4. Open Transactions and select a payment.
5. Show the Intent and expected amount.
6. Show Razorpay payment capture evidence.
7. Show settlement recon gross, fee, tax, and net.
8. Show bank statement match or intentionally missing bank credit.
9. Open Payment Gaps and show the missing link.
10. Open Outcome Review and show the evidence-backed recommendation.
11. Open Overview and show totals and freshness.
12. Open Intelligence and show anomaly/recommendation output.
13. Open Cash Forecast and show future incoming/outgoing money.
14. Ask Ask Zord, "Did this payment reach the bank?"
15. Show an answer with citations and correct uncertainty.
16. Open Audit Evidence and verify the hash/Merkle proof.

26. Final page list

#	Page	Route	Priority
0	Login/onboarding	/login , /onboarding	P0
1	Overview dashboard	/dashboard	P0
2	Data imports	/data-imports	P0
3	Transactions	/transactions	P0
4	Transaction detail/proof	/transactions/:id	P0
5	Intents/orders	/intents	P0
6	Intent detail	/intents/:id	P0
7	Settlements	/settlements	P0

8	Settlement detail	/settlements/:id	P0
9	Payment gaps	/payment-gaps	P0
10	Gap detail	/payment-gaps/:id	P0
11	Reconciliation control center	/reconciliation	P1
12	Outcome review	/outcome-review	P1
13	Outcome detail	/outcome-review/:id	P1
14	Intelligence center	/intelligence	P1
15	Cash forecast	/forecasts/cash	P1
16	Order forecast	/forecasts/orders	P1
17	Ask Zord	/ask	P1
18	Audit evidence	/evidence	P1
19	Connector management	/connectors	P0/P1
20	Backfill operations	/operations/backfills	P1
21	Refunds	/refunds	P2
22	Disputes	/disputes	P2
23	Payment Links	/payment-links	P2
24	Invoices	/invoices	P2
25	Subscriptions	/subscriptions	P2
26	Settings/access	/settings	P1/P2

The frontend should be built around the merchant journey, not around individual provider APIs. The merchant should always be able to move from a summary number to a transaction, from a transaction to a settlement, from a settlement to a bank match, and from an unresolved match to evidence-backed review.